

1. Network Troubleshooting Workflow

When network fails check:

Step 1 – Interface

`ip addr`

Step 2 – Gateway

`ip route`

Step 3 – Ping gateway

`ping 192.168.1.1`

Step 4 – Ping internet

`ping 8.8.8.8`

Step 5 – Check DNS

`dig google.com`

2. Top Networking Commands in Linux

1. **ip** (MOST IMPORTANT – Modern Networking Tool)

Purpose

Manages:

- IP addresses
- Network interfaces
- Routing tables

Why we use it

`ip` replaces older tools (`ifconfig`, `route`). It is:

- More powerful
- Script-friendly
- Default in modern Linux (RHEL/CentOS 9)

Package

iproute

Commands

ip addr show

ip route

ip link show

Example

ip addr show eth0

Sample Output

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500

inet 192.168.1.100/24 brd 192.168.1.255

Breakdown

- **eth0** → Interface name
- **UP** → Interface is active
- **inet 192.168.1.100/24** → IP address
- **/24** → Subnet mask

👉 If no **inet** → No IP assigned

✓ Check if server has IP

👉 Real case: Website down → verify server IP exists

2. **ifconfig** (Legacy Tool)

Purpose

Configure and display network interfaces

Why we use it

- Used in old systems
- Still appears in interviews & legacy scripts

Package

net-tools

Commands

ifconfig

ifconfig eth0 up

Example

sudo ifconfig eth0 down

sudo ifconfig eth0 up

Output

eth0: flags=4163<UP,BROADCAST,RUNNING>

inet 192.168.1.100

Breakdown

- **UP** → Interface running
- **inet** → IP address

👉 Legacy version of **ip**

👉 Real case: Interface stuck → restart it

3. ping

Purpose

Check connectivity between systems using ICMP

Why we use it

- First step in troubleshooting

- Checks if host is reachable

Package

iputils

Commands

ping google.com

ping -c 4 8.8.8.8

Example

ping -c 4 8.8.8.8

Output

64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=20 ms

Breakdown

- **icmp_seq** → Packet number
- **ttl** → Time to live
- **time** → Latency

👉 High time = slow network

👉 No reply = unreachable

👉 If works → Internet OK

👉 If fails → network issue

4. traceroute

Purpose

Shows path packets take to reach destination

Why we use it

- Identify where delay or failure occurs

Package

traceroute

Commands

tracert google.com

Example

tracert api.company.com

Output

```
1 192.168.1.1 1 ms
2 10.10.0.1 5 ms
3 8.8.8.8 20 ms
```

Breakdown

- Each line = network hop
- **ms** = latency

👉 Delay at hop = bottleneck

👉 Helps detect slow ISP or routing issue

5. ss (Modern Socket Tool)

Purpose

Displays open ports and active connections

Why we use it

- Faster replacement of **netstat**
- Used to check running services

Package

iproute

Commands

ss -tulnp

Example

ss -tulnp | grep 80

Output

LISTEN 0 128 0.0.0.0:22 users:(("sshd",pid=1234))

Breakdown

- **LISTEN** → Service is active
- **:22** → Port number
- **sshd** → Process name

👉 No entry = service not running

👉 Check if web server is running

6. **netstat** (Legacy)

Purpose

Shows:

- Open ports
- Connections
- Routing table

Why we use it

- Older systems
- Still widely used

Package

net-tools

Commands

netstat -tulnp

Example

netstat -tulnp | grep 22

Output

tcp 0 0 0.0.0.0:80 LISTEN 1234/nginx

Breakdown

- `tcp` → Protocol
- `:80` → HTTP port
- `LISTEN` → Waiting for connection

👉 Verify SSH is running

7. `curl`

Purpose

Transfer data from/to servers (HTTP, HTTPS, APIs)

Why we use it

- Test APIs
- Check web services
- Debug microservices

Package

`curl`

Commands

```
curl http://example.com
curl -I http://example.com
```

Example

```
curl http://localhost:8080/health
```

Output

```
{"status":"UP"}
```

Breakdown

- JSON response → API working
- HTML output → website working

👉 If timeout → service/network issue

👉 Output confirms API health

🔥 8. **wget**

✅ **Purpose**

Download files from internet

🎯 **Why we use it**

- Install software
- Download backups/scripts

📦 **Package**

wget

🔧 **Commands**

wget URL

🧪 **Example**

wget <https://example.com/file.tar.gz>

📦 **Output**

Saving to: 'file.zip'

100% [=====]

🧠 **Breakdown**

- Shows download progress
 - Errors → network or URL issue
-

🔥 9. **dig**

✅ **Purpose**

Advanced DNS lookup tool

Why we use it

- Debug DNS issues
- Check domain → IP mapping

Package

bind-utils

Commands

dig google.com

dig google.com +short

Example

dig api.company.com

Output

ANSWER SECTION:

google.com. 300 IN A 142.250.182.206

Breakdown

- **google.com** → domain
- **A** → IPv4 record
- **IP** → resolved address

👉 No answer = DNS issue

10. nslookup

Purpose

Query DNS servers

Why we use it

- Simple DNS debugging

Package

bind-utils

Commands

nslookup google.com

Example

nslookup google.com 8.8.8.8

Output

Server: 8.8.8.8

Address: 142.250.182.206

Breakdown

- **Server** → DNS server used
 - **Address** → resolved IP
-

11. host

Purpose

Quick DNS lookup

Why we use it

- Fast alternative to dig/nslookup

Package

bind-utils

Commands

host google.com

Example

host api.example.com

Output

google.com has address 142.250.182.206

Breakdown

Simple DNS result

12. tcpdump (Advanced)

Purpose

Capture and analyze network packets

Why we use it

- Deep debugging
- Verify traffic reaching server

Package

tcpdump

Commands

```
tcpdump -i eth0
```

```
tcpdump port 80
```

Example

```
sudo tcpdump -i eth0 port 8080
```

Output

```
IP 192.168.1.5.12345 > 192.168.1.10.80: Flags [S]
```

Breakdown

- Source IP → 192.168.1.5
- Destination → 192.168.1.10:80
- [S] → SYN (connection start)

 Used to confirm traffic flow

 Shows live traffic

13. nmap

Purpose

Scan open ports and services

Why we use it

- Security checks
- Firewall validation

Package

nmap

Commands

nmap IP

nmap -p 80,443 IP

Example

nmap -p 22,80 localhost

Output

PORT STATE SERVICE

22/tcp open ssh

80/tcp open http

Breakdown

- **open** → accessible
- **closed** → blocked

 Helps check firewall

14. nmcli

Purpose

Control NetworkManager from CLI

Why we use it

- Manage network without GUI
- Used in servers & automation

Package

NetworkManager

Commands

`nmcli device status`

`nmcli connection show`

Example

`nmcli device disconnect eth0`

`nmcli device connect eth0`

Output

DEVICE	TYPE	STATE	CONNECTION
eth0	ethernet	connected	Wired

Breakdown

- **connected** → network active
- **disconnected** → no network

Example to set IP address

`ip a` [Note down the Network I/F name]

`nmcli connection modify enp0s3 ipv4.addresses 192.168.31.10/24 ipv4.gateway 192.168.31.1`

`ipv4.dns 8.8.8.8 ipv4.method manual`

`nmcli connection down enp0s3`

`nmcli connection up enp0s3`

`nmcli connection show`

`nmcli device status`

`ip a` [New Static IP will be assigned to Network I/F]

15. route

Purpose

View routing table

Why we use it

- Legacy alternative to `ip route`

Package

net-tools

Commands

`route -n`

Example

`route -n`

Output

```
Destination Gateway
0.0.0.0      192.168.1.1
```

Breakdown

- `0.0.0.0` → default route
- `192.168.1.1` → gateway

👉 Missing → no internet

👉 Check default gateway

16. `arp`

Purpose

View ARP table (IP → MAC mapping)

Why we use it

- Debug local network issues

Package

net-tools

Commands

arp -a

Example

arp -a

Output

192.168.1.1 at 00:1a:2b:3c:4d:5e

Breakdown

- IP → router
- MAC → hardware address

👉 Used in LAN debugging

👉 Check router MAC address

17. iftop

Purpose

Monitor bandwidth usage per connection

Why we use it

- Detect high traffic
- Identify bandwidth hogs

Package

iftop

Commands

sudo iftop

Example

👉 Shows:

- Source IP

- Destination IP
- Speed (Mb/s)

Output

192.168.1.10 => 8.8.8.8 10Mb/s

Breakdown

- Source → Destination
- Bandwidth usage

 Detect heavy traffic

18. nload

Purpose

Monitor total network traffic

Why we use it

- Simple real-time bandwidth graph

Package

nload

Commands

nload

Output

Incoming: 5 Mb/s

Outgoing: 2 Mb/s

Breakdown

- Total traffic usage
-

19. **hostname**

Purpose

Show/set system hostname

Why we use it

- Identify server in cluster
- Debug networking

Package

util-linux

Commands

hostname
hostname -l

Example

hostname -l

Output

prod-server
192.168.1.100

Breakdown

- Hostname → system name
- IP → system address

 Shows system IP

20. **mtr** (BEST DEBUG TOOL)

Purpose

Combines ping + traceroute

Why we use it

- Detect packet loss
- Measure latency continuously

Package

mtr

Commands

mtr google.com

Example

mtr -r -c 10 [google.com](https://www.google.com)

Output

```
HOST      LOSS%  AVG
192.168.1.1  0%    1ms
8.8.8.8     0%    20ms
```

Breakdown

- **LOSS%** → packet loss
- **AVG** → latency

👉 Packet loss = network issue

👉 Shows:

- Packet loss %
- Latency per hop

FINAL SUMMARY (VERY IMPORTANT)

Category	Commands
Interface & IP	ip, ifconfig, nmcli
Connectivity	ping, traceroute, mtr
Ports & Services	ss, netstat
DNS	dig, nslookup, host

Traffic Analysis	tcpdump, iftop, nload
------------------	-----------------------

Download/API	curl, wget
--------------	------------

Routing	ip route, route
---------	-----------------

Network Mapping	arp
-----------------	-----

Security	nmap
----------	------

System Info	hostname
-------------	----------
